Dossier Technique ECF — Vite & Gourmand

TP Développeur Web et Web Mobile (Studi) · Antoine Banzet · 2026

Informations obligatoires & liens du projet

- **Dépôt GitHub :** https://github.com/antoinebaban-sketch/ECF-DWWM
- **Outil de gestion de projet :** GitHub — jalons (sprints) et issues (tâches)
- **Déploiement (application en ligne) :** https://ecf-dwwm.onrender.com
- **Identifiants administrateur :** `admin@viteetgourmand.fr` / `Admin2026!`

Partie 1 — Analyse des besoins

1.1 Résumé du projet

Vite & Gourmand est un traiteur haut de gamme installé à Bordeaux depuis 1999. Après plus de 25 ans d'activité, l'entreprise dispose d'une clientèle fidèle mais gère encore sa prise de commande par téléphone et par email. Le projet consiste à numériser l'ensemble de la chaîne : catalogue, commande, livraison et gestion interne.

L'application web et web mobile permet de consulter une carte de menus filtrables par régime alimentaire et par allergène, de passer commande en ligne avec calcul automatique de la tarification et des frais de livraison, de déposer un avis soumis à modération, et donne au personnel des espaces de gestion dédiés. Le système distingue **quatre profils** : le Visiteur (non authentifié, pages publiques uniquement), le Client, l'Employé et l'Administrateur.

L'architecture repose sur une **double base de données** — MySQL pour les données transactionnelles, MongoDB pour l'analyse décisionnelle de l'espace administrateur — une **API REST JSON en PHP 8.3, entièrement orientée objet** (architecture MVC : contrôleurs, modèles, repositories, organisés derrière un *Front Controller*), et un front-end HTML / CSS / JavaScript sans framework, conforme aux normes RGAA / WCAG 2.1 niveau AA.

1.2 Spécifications fonctionnelles

Module Client — Inscription (consentement RGPD obligatoire, case non pré-cochée), connexion, réinitialisation du mot de passe par lien à usage unique. Consultation dynamique du catalogue avec filtres stricts par régime et par allergène. Passage de commande avec choix de livraison au tarif kilométrique. Modification et annulation libres tant que la commande n'est pas passée au statut « acceptée ». Suivi du statut après acceptation, avec historique daté. Dépôt d'un avis sur une commande terminée. Demande de facture. Export de ses données personnelles (RGPD Art. 20) et suppression de compte (RGPD Art. 17, par pseudonymisation).

Module Employé — Gestion CRUD complète des menus, plats, régimes, allergènes et horaires. Gestion des commandes : changement de statut parmi les 8 étapes du cycle de vie, filtre par statut et par client. L'annulation ou la modification d'une commande après acceptation impose la saisie d'un motif et d'un mode de contact. Modération des avis clients avant publication.

Module Administration — Toutes les prérogatives de l'employé, plus la création et la désactivation de comptes employés (le mot de passe n'est jamais transmis en clair : l'employé définit le sien via un lien d'activation à usage unique). Accès au tableau de bord analytique : chiffre d'affaires par mois et par menu (calculé en SQL), nombre de commandes par menu (calculé en MongoDB, restitué sous forme de graphique comparatif).

Cycle de vie d'une commande : `en_attente` → `acceptée` → `en_preparation` → `en_cours_livraison` → `livrée` → `retour_materiel` → `terminée` ; `annulée` possible avant acceptation (par le client) ou à tout moment (par le personnel, avec motif).

Règles métier de l'énoncé : réduction automatique de 10 % lorsque le nombre de convives atteint le minimum requis majoré de 5 ; frais de livraison de 5 € forfaitaires + 0,59 €/km hors de Bordeaux ; retour du matériel loué sous 10 jours ouvrés, sous peine d'une pénalité de 600 € ; validation obligatoire des avis avant publication ; interdiction absolue de créer un compte administrateur depuis l'interface publique.

Partie 2 — Spécifications techniques

2.1 Choix de la stack technique

Composant	Technologie	Justification
Backend / API	PHP 8.3 · PDO · API REST JSON POO, architecture MVC · autoload PSR-4 via Composer (namespace `App\`)	Hébergement PHP standard, maîtrise totale de la logique et de la sécurité, sans framework imposé. L'API est entièrement écrite en classes : chaque responsabilité (routage, contrôle d'accès, règles métier, accès aux données, emails) a sa propre classe (voir 2.3).
SGBDR	MySQL 8+ · InnoDB · utf8mb4	Propriétés ACID, clés étrangères et contraintes d'intégrité référentielle indispensables pour une base de commandes ; relations Plusieurs-à-Plusieurs via tables de liaison.
NoSQL	MongoDB · pilote `ext-mongodb` (pas de paquet Composer dédié)	Base non relationnelle exigée par l'énoncé ; *aggregation pipeline* pour découpler le calcul décisionnel du transactionnel.
Frontend	HTML5 · CSS3 (Custom Properties) · JavaScript vanilla (ES6+) un fichier `js/pages/<page>.js` par page, aucun script inline	Compatibilité multi-support, conformité RGAA sans dépendance, aucune étape de build. Le HTML ne contient aucun code JavaScript, ce qui permet une *Content-Security-Policy* stricte (voir 2.7).
Authentification	Sessions PHP (cookie `HttpOnly` / `Secure`) · bcrypt cost 12	Mécanisme natif de PHP, aucun code cryptographique à écrire, cookie non lisible en JavaScript.
Emails	`mail()` natif en local · API HTTP Brevo en production	Un conteneur Docker n'embarque pas de serveur mail ; l'API HTTP de Brevo contourne cette limite en production.

2.2 Environnement de développement et de déploiement

Développement local — Stack XAMPP (Apache, PHP 8.x, MySQL 8+), Composer pour l'autoload PSR-4 (`composer install` dans `api/`), base MongoDB hébergée sur MongoDB Atlas (cluster gratuit). Gestion de version : Git et dépôt GitHub. Organisation des branches : `main` (code stable / production), `develop` (intégration), et une branche `feature/*` par fonctionnalité, fusionnée dans `develop` avec `--no-ff` après test ; `develop` est fusionnée dans `main` pour marquer les versions stables. Messages de commit atomiques et préfixés (`feat` , `fix` , `chore` , `docs`).

Déploiement en production — Image Docker (base `php:8.3-apache` — PHP 8.1 n'étant plus maintenu en sécurité depuis fin 2025) déployée sur **Render**. Le script `docker-entrypoint.sh` adapte le port d'écoute d'Apache à la variable `$PORT` imposée par l'hébergeur. Base MySQL managée sur **Aiven** (connexion SSL via le certificat `api/aiven-ca.pem` , activé automatiquement dès que l'hôte de base n'est plus `localhost`). MongoDB sur **Atlas**. Emails via l'API HTTP de **Brevo**.

Isolation des variables sensibles — Le fichier `api/.env` n'est jamais versionné (listé dans `.gitignore`) ; un modèle `api/.env.example` documente les variables attendues. En production, les valeurs sont injectées par le tableau de bord de Render.

Étapes de la mise en production

1. Créer la base MySQL sur Aiven et importer `vite_et_gourmand.sql` via la console Aiven ; récupérer l'hôte, le port, l'utilisateur, le mot de passe et le certificat SSL.
2. Créer le cluster MongoDB sur Atlas et récupérer la chaîne de connexion ; créer la clé API Brevo.
3. Sur Render : créer un service « Web Service » lié au dépôt GitHub, type *Docker* (le `Dockerfile` à la racine de `projet/` est détecté automatiquement).
4. Renseigner les variables d'environnement dans le tableau de bord Render (`DB_HOST` , `DB_PORT` , `DB_NAME` , `DB_USER` , `DB_PASS` , `MONGO_URI` , `MONGO_DB` , `BREVO_API_KEY` , `APP_URL` , `MAIL_FROM` , `MAIL_NAME`).
5. Déclencher le premier déploiement : Render construit l'image (`docker build` , qui exécute aussi `composer install` pour générer l'autoload), l' `ENTRYPOINT` adapte le port d'Apache à `$PORT` , puis lance `apache2-foreground` .
6. Vérifier : chargement de la page d'accueil, réponse de `/api/menus` , connexion d'un compte de test.

Procédure de mise à jour — Un `push` sur la branche déployée déclenche automatiquement la reconstruction et le redéploiement de l'image par Render. Les évolutions de schéma de base de données sont appliquées manuellement (phpMyAdmin en local, console Aiven en production).

Expéditeur des emails transactionnels — L'API Brevo exige que l'adresse `MAIL_FROM` corresponde à un expéditeur vérifié du compte (domaine authentifié par DNS, ou adresse individuelle validée par lien de confirmation) : tout envoi depuis une adresse non vérifiée est rejeté par Brevo sans remonter d'erreur côté application. Le domaine `viteetgourmand.fr` étant fictif, il ne peut pas être authentifié ; `MAIL_FROM` utilise donc l'adresse `antoinebaban@gmail.com`, vérifiée dans le compte Brevo, comme expéditeur technique — le nom affiché reste « Vite & Gourmand » via la variable `MAIL_NAME`.

2.3 Architecture logicielle de l'API — programmation orientée objet (MVC)

L'API ne contient plus aucun fichier de fonctions : tout le code serveur est organisé en **classes** (dossier `api/src/`, namespace `App\`, chargement automatique PSR-4 par Composer). Le fichier `api/index.php` se limite à charger l'autoload, la configuration et la table des routes (`api/routes.php`), puis à confier la requête au `Kernel`. Le découpage suit le patron **MVC** : la *Vue* est le front-end HTML / JavaScript qui consomme le JSON, le *Contrôleur* orchestre, le *Modèle* porte les règles métier et les *Repositories* l'accès aux données.

Couche	Classes	Responsabilité
HTTP	`Kernel`, `Router`, `Request`, `JsonResponse`, `HttpException`	CORS et session, routage déclaratif (`GET menus/{id}` → `MenuController::show`), lecture de la requête, envoi de la réponse. Le `Kernel` est le seul endroit qui convertit les exceptions en réponses JSON.
Contrôleurs	`Controller` (abstraite) + 11 contrôleurs : `Auth`, `Menu`, `Plat`, `Theme`, `Referentiel`, `Horaire`, `Commande`, `Avis`, `Utilisateur`, `Admin`, `Contact`	Contrôle des droits, validation des entrées, appel du modèle et du repository, construction de la réponse. Aucun SQL, aucun calcul métier.
Modèles	`Commande`, `Utilisateur`, `Avis`	Règles métier : calcul du prix (remise de 10 %, frais de livraison), droit du client à modifier sa commande, délai de retour du matériel, note comprise entre 1 et 5, comptes non supprimables ou non désactivables.
Énumérations	`StatutCommande`, `StatutAvis`, `Role`	Valeurs autorisées typées (enums PHP 8.1) : un statut invalide est rejeté par `StatutCommande::tryFrom()`.
Repositories	`Repository` (abstraite) + `Commande`, `Menu`, `Plat`, `Referentiel`, `Avis`, `Utilisateur`, `Statistique`	Tout le SQL de l'application, uniquement en requêtes préparées ; transactions via `transaction()` (commit ou rollback automatique).
Services	`Database`, `MongoConnection`, `AuthService`, `PasswordPolicy`, `Sanitizer`, `Mailer`, `Notifier`, `Config`	Connexions (PDO unique par requête, MongoDB avec dégradation gracieuse), session et rôles, politique de mot de passe et bcrypt, nettoyage des entrées, catalogue des emails transactionnels.

Principes objet mis en œuvre

- **Encapsulation** — propriétés `private readonly` ; seule `AuthService` lit ou écrit `$_SESSION`, seule `Database` ouvre la connexion PDO.
- **Héritage et classes abstraites** — chaque contrôleur hérite de `Controller` (accès à `AuthService`, construction des réponses), chaque repository de `Repository` (PDO, transactions, requêtes préparées).
- **Exceptions métier** — une erreur attendue (droit insuffisant, champ manquant, ressource absente) est levée sous forme de `HttpException` n'importe où dans le code ; le `Kernel` la transforme en réponse `{"error": "…"}` avec le bon code HTTP. Toute autre erreur produit un 500 générique, sans détail technique exposé au client.
- **Typage strict** — `declare(strict_types=1)`, types de retour, propriétés en lecture seule, énumérations.

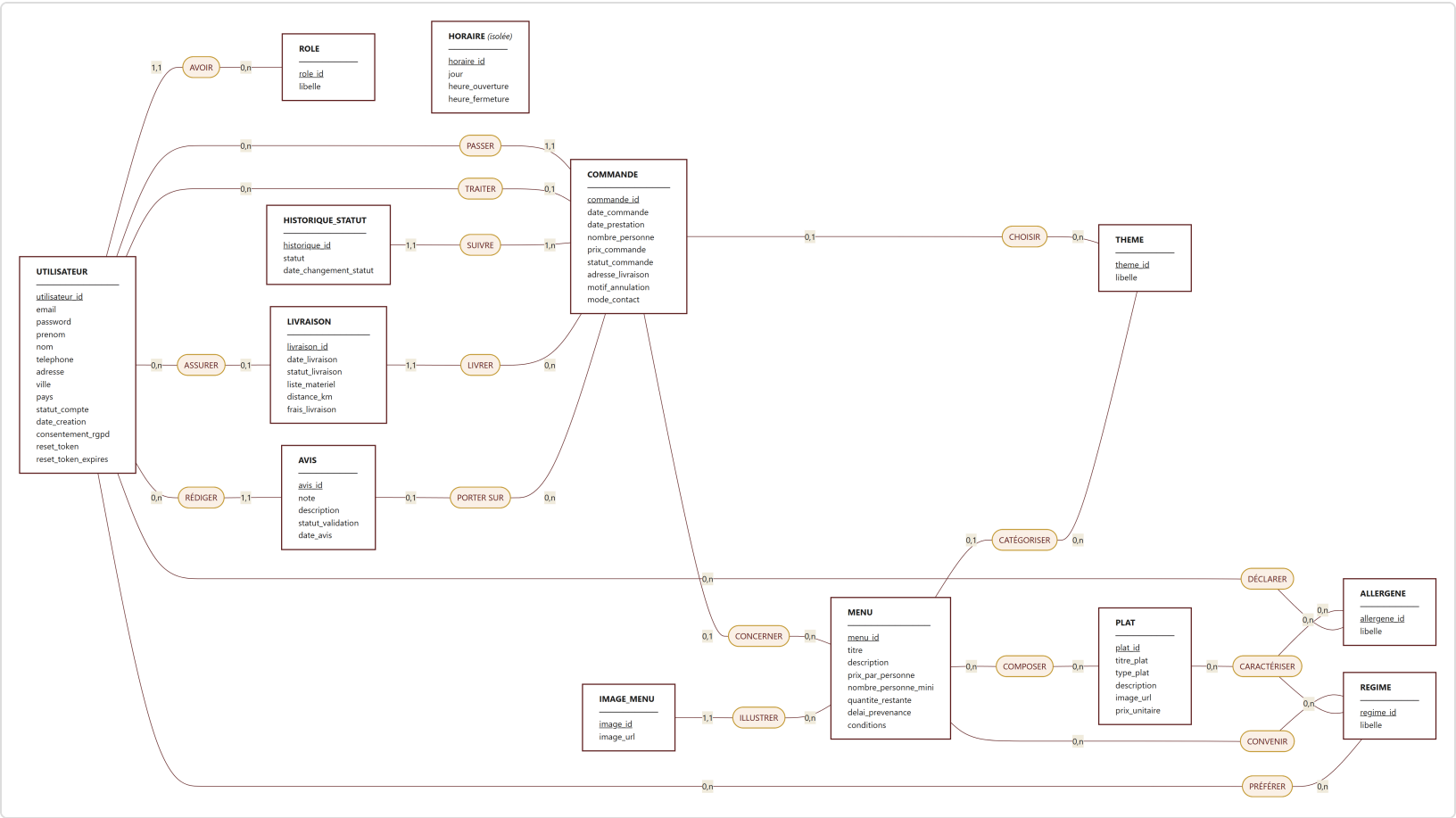
```
// Modèle : la règle métier vit dans l'entité, pas dans le contrôleur
final class Commande {
    public function modifiableParClient(): bool {
        return $this->statut === StatutCommande::EnAttente;
    }
}

// Contrôleur : droits, appel du repository et du modèle, réponse
public function annuler(Request $request, int $id): JsonResponse {
    $user = $this->auth->requireUser();
    $row  = $this->commandes->findByIdAndClient($id, (int) $user['id'])
         ?? throw HttpException::notFound('Commande introuvable');
    if (!Commande::fromRow($row)->modifiableParClient()) {
        throw new HttpException('Cette commande ne peut plus être annulée');
    }
    $this->commandes->annuler($id);
    return $this->message('Commande annulée');
}
```

Non-régression — Le passage du code procédural aux classes a été vérifié par un scénario de 144 requêtes couvrant toutes les routes (cas nominaux, erreurs de validation, droits refusés), rejoué sur une copie de la base avant et après la migration : réponses JSON et état final de la base identiques. Le contrat de l'API n'a donc pas changé pour le front-end.

2.4 Conception de la base de données relationnelle (MySQL)

La modélisation a suivi trois étapes : MCD (entités, associations, cardinalités, indépendamment de toute technique) → MLD (traduction relationnelle : tables, colonnes, placement des clés primaires et étrangères, relations N-N transformées en tables de liaison) → DDL (implémentation).



MCD (méthode Merise) — 13 entités, associations nommées par un verbe, cardinalités (min,max) côté entité ; pas de types ni de clés étrangères à ce niveau.

Le MCD décrit les **données** ; il est distinct du diagramme de classes (2.8), qui décrit le **code**. Au passage au MLD, les associations de cardinalités `0,n – 0,n` deviennent des tables de liaison (`menu_composition` , `menu_regime` , `utilisateur_regime` , `utilisateur_allergene` , et l'association ternaire

CARACTÉRISER → `plat_allergene_regime`), et les associations hiérarchiques `1,n` (cardinalité `1,1` ou `0,1` côté entité fille) deviennent des clés étrangères : 18 tables au total. Le modèle ne contient aucune relation 1-1 : une commande peut ainsi avoir plusieurs livraisons (LIVRER) et plusieurs avis (PORTER SUR).

Choix techniques — Moteur **InnoDB** (et non MyISAM) pour les clés étrangères et les transactions ACID. Encodage **utf8mb4** (et non `utf8`, limité à 3 octets et incompatible avec certains caractères). Normalisation **3NF** : la table `commande` stocke `client_id` (clé étrangère) et non le nom ou l'email du client, qui n'ont donc pas à être dupliqués ni synchronisés.

Décisions structurantes

- **Prix historisé** — Le prix unitaire et le montant total sont figés au moment de la commande, jamais recalculés à l'affichage : principe comptable de la pièce justificative. Si le tarif d'un menu évolue, la facture initiale du client reste inchangée.
- **Pseudo-suppression des utilisateurs** — Une colonne `statut_compte` (0/1) désactive l'accès sans supprimer physiquement la ligne (ce qui casserait les clés étrangères des commandes passées). Le droit à l'effacement RGPD est traité par pseudonymisation (voir Partie 4).
- **Tables de liaison N-N** — Le schéma compte 5 tables de liaison : menu ↔ plat, menu ↔ régime, utilisateur ↔ régime, utilisateur ↔ allergène, et la table ternaire plat ↔ allergène ↔ régime (`plat_allergene_regime`).

```
CREATE TABLE `menu_regime` (
    `menu_id`   INT NOT NULL,
    `regime_id` INT NOT NULL,
    PRIMARY KEY (`menu_id`, `regime_id`),
    CONSTRAINT `fk_mr_menu`   FOREIGN KEY (`menu_id`)   REFERENCES `menu`(`menu_id`)   ON DELETE CASCADE,
    CONSTRAINT `fk_mr_regime` FOREIGN KEY (`regime_id`) REFERENCES `regime`(`regime_id`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Le `ON DELETE CASCADE` sur `fk_mr_menu` nettoie automatiquement les liaisons lorsqu'un menu est supprimé. L'absence volontaire de cascade sur `fk_mr_regime` (comportement par défaut, `RESTRICT`) **bloque** la suppression d'un régime encore rattaché à des menus — garantie de cohérence pour les clients ayant des contraintes alimentaires.

2.5 Indexation et optimisation des performances

Sans index, MySQL effectue une lecture séquentielle complète (*Full Table Scan*), de complexité linéaire *O(n)*. Les index explicites construisent des arbres B-tree ramenant la recherche à une complexité logarithmique *O(log n)*. Des index ont été créés sur les colonnes sollicitées par les clauses `WHERE`, `JOIN` et `ORDER BY` de l'API :

```
-- Espace client et suivi des commandes
CREATE INDEX idx_commande_client  ON `commande`(`client_id`);
CREATE INDEX idx_commande_statut  ON `commande`(`statut_commande`);
CREATE INDEX idx_commande_date    ON `commande`(`date_commande`);
CREATE INDEX idx_commande_menu    ON `commande`(`menu_id`);
-- Modération des avis
CREATE INDEX idx_avis_statut      ON `avis`(`statut_validation`);
CREATE INDEX idx_avis_client      ON `avis`(`client_id`);
-- Contrôle d'accès (RBAC)
CREATE INDEX idx_utilisateur_role ON `utilisateur`(`role_id`);
```

2.6 Base de données NoSQL (MongoDB)

Conformément à l'énoncé, MongoDB est exploité **exclusivement** pour l'analyse décisionnelle du tableau de bord administrateur : la visualisation du nombre de commandes par menu. Son usage n'a volontairement pas été étendu à un système de logs ou à la gestion de sessions, pour rester dans le périmètre demandé et garder une séparation nette des responsabilités.

Chaque commande enregistre, **dès sa création**, un document dans la collection `commandes` :

```
{
  "_id": ObjectId("6693f1a2e4b0a123456789ab"),
  "commande_id": 157,
  "menu_id": 8,
  "menu_titre": "Menu Prestige Bordeaux",
  "montant": 2550.00,
  "date": "2026-07-14"
}
```

Le comptage s'appuie sur un *aggregation pipeline*, équivalent d'un `GROUP BY` + `COUNT`. Le regroupement se fait sur `$menu_id` (et non sur le titre) pour éviter toute collision entre deux menus homonymes ; le titre est récupéré via `$first` :

```
db.commandes.aggregate([
  { $group: {
      _id: "$menu_id",
      menu_titre: { $first: "$menu_titre" },
      nb_commandes: { $sum: 1 }
  }},
  { $sort: { nb_commandes: -1 } }
]);
```

Séparation des responsabilités — MongoDB ne fait que du comptage (`nb_commandes`). Le calcul financier du chiffre d'affaires reste traité par MySQL via une requête dédiée, garantissant l'étanchéité entre gestion comptable et outil décisionnel.

Dégradation gracieuse — Si `MONGO_URI` n'est pas renseignée ou si le serveur MongoDB est indisponible, l'application entière continue de fonctionner : prise de commande et navigation restent 100 % opérationnelles. Seul le bloc graphique de l'espace administration affiche « Aucune donnée (MongoDB non configuré ?) ». Un service d'analyse secondaire ne doit jamais bloquer le cœur de métier.

2.7 Mécanismes de sécurité — grille OWASP Top 10

Menace OWASP	Risque	Mécanisme déployé
A01 — Broken Access Control	Appel non autorisé d'endpoints API	Session PHP (cookie `HttpOnly`) + contrôle de rôle systématique côté serveur (`AuthService::requireRole()` / `requireStaff()` , appelé en tête de chaque méthode de contrôleur), jamais uniquement côté frontend.
A02 — Cryptographic Failures	Compromission des mots de passe	Hachage `bcrypt` cost 12 via `password_hash()` (classe `PasswordPolicy`) (~250 ms par hachage, débit d'un attaquant fortement limité). Jamais de stockage en clair.
A03 — Injection SQL	Injection de code via les formulaires	Tout le SQL est regroupé dans les repositories ; 100 % des requêtes touchant une donnée utilisateur passent par des requêtes préparées PDO avec paramètres liés (`PDO::ATTR_EMULATE_PREPARES = false` : vraies requêtes préparées côté serveur).
A03 — Cross-Site Scripting (XSS)	Injection de scripts via les champs libres	Triple protection : `strip_tags()` côté serveur avant stockage (`Sanitizer`), échappement HTML (`escapeHtml()`) côté client à l'affichage de toute donnée saisie par un utilisateur, et *Content-Security-Policy* `script-src 'self'` : les pages ne contenant aucun script inline, un script injecté ne serait pas exécuté par le navigateur.
A05 — Security Misconfiguration	Fuite d'informations techniques	`display_errors` désactivé ; le `Kernel` intercepte toute exception non prévue, la journalise côté serveur (`error_log`) et ne renvoie qu'un message générique. Fichier `.env` jamais versionné.

Périmètre de l'échappement — Il est systématique sur les données saisies par le public (avis, formulaires, profil). Les libellés de catalogue affichés côté public (titres de menus, plats, régimes, allergènes) sont gérés exclusivement par le personnel via des routes protégées (`AuthService::requireStaff()`) ; le risque associé se limite donc au cas d'un compte staff compromis, hors du périmètre de menace principal.

Ne sont pas implémentés, n'étant pas demandés par l'énoncé : reCAPTCHA, double authentification, limitation de débit anti-bruteforce.

2.8 Modélisation UML

Diagramme de cas d'utilisation — Il cartographie les frontières du système et le cloisonnement des droits. Le Visiteur, non authentifié, n'a aucun droit spécifique à restreindre ; le diagramme se concentre sur les trois profils authentifiés (Client, Employé, Administrateur), avec une relation d'héritage : l'Administrateur reprend tous les cas d'usage de l'Employé.

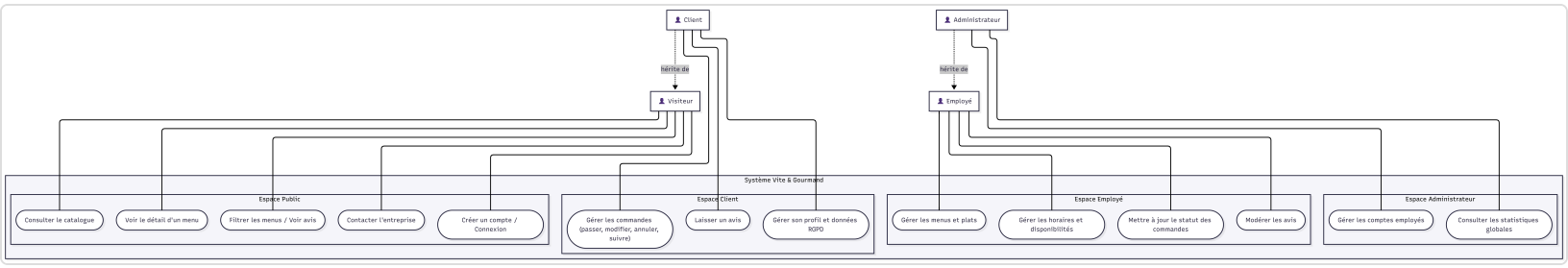
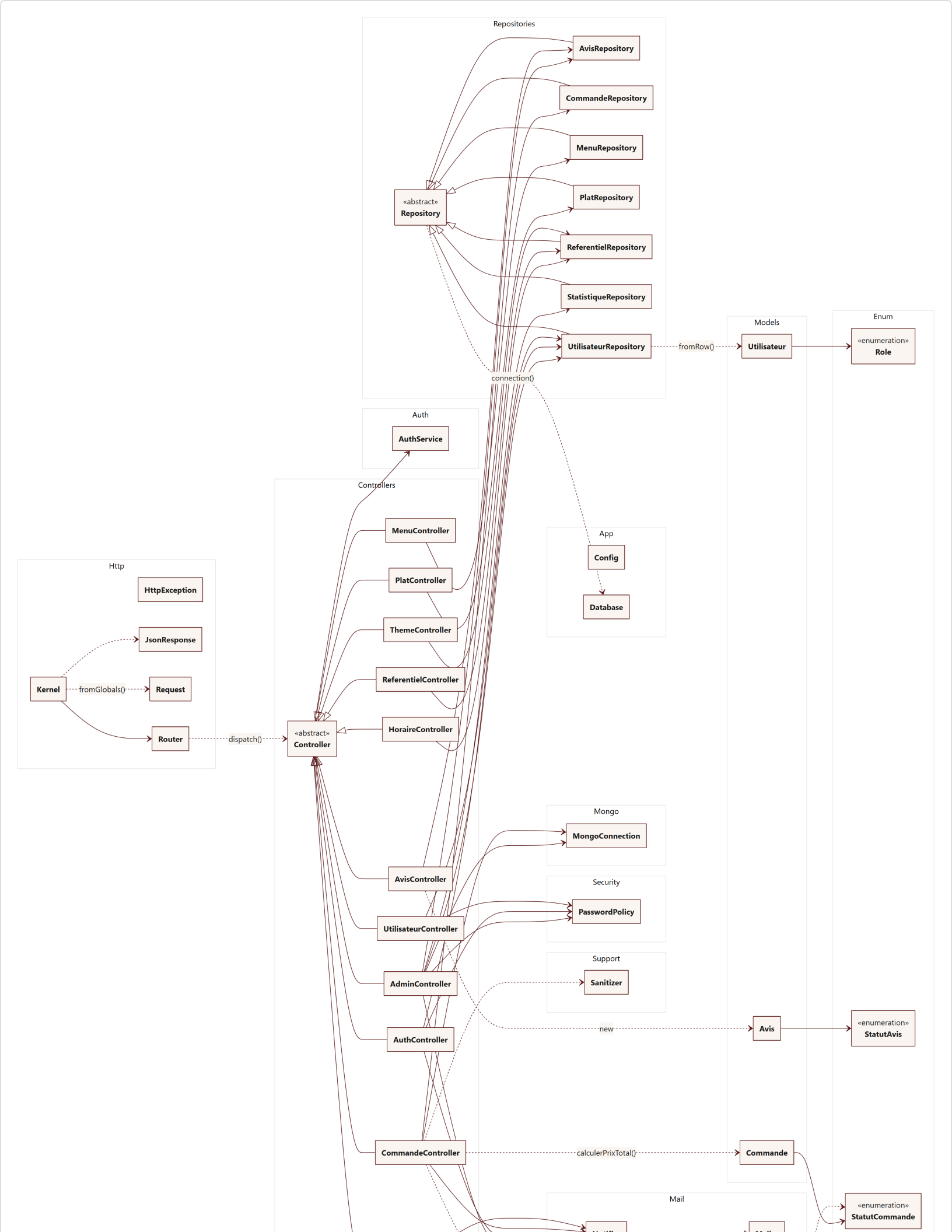


Diagramme de cas d'utilisation — fonctionnalités par acteur.

Diagrammes de classes — Ils décrivent le code de l'API (et non la base de données, décrite par le MCD en 2.4). Ils sont **générés à partir du code réel** par réflexion PHP (`docs/diagrammes/sources/classes.php`) : le schéma ne peut pas diverger des classes écrites. La vue d'ensemble montre les couches et l'héritage ; le second schéma détaille attributs et méthodes du domaine « commande » : contrôleur, repository, modèle et énumération, avec leurs classes mères.


Repositories
AvisRepository
CommandeRepository
MenuRepository
PlatRepository
«abstract»
Repository
ReferentielRepository
StatistiqueRepository
UtilisateurRepository
fromRow()
Models
Utilisateur
Enum
«enumeration»
Role
connection()
Auth
AuthService
Controllers
MenuController
PlatController
ThemeController
ReferentielController
HoraireController
App
Config
Database
Http
HttpException
JsonResponse
Kernel
fromGlobals()
Request
Router
dispatch()
«abstract»
Controller
Mongo
MongoConnection
AvisController
Security
PasswordPolicy
UtilisateurController
Support
Sanitizer
AdminController
new
Avis
«enumeration»
StatutAvis
AuthController
CommandeController
calculerPrixTotal()
Commande
Mail
«enumeration»
StatutCommande

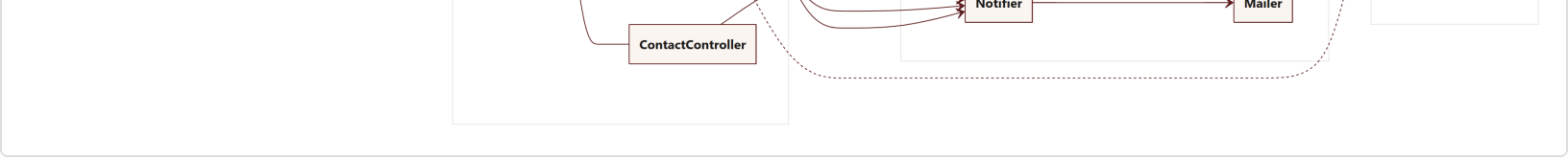


Diagramme de classes — vue d'ensemble : HTTP → contrôleurs → repositories / modèles / énumérations, services transverses.

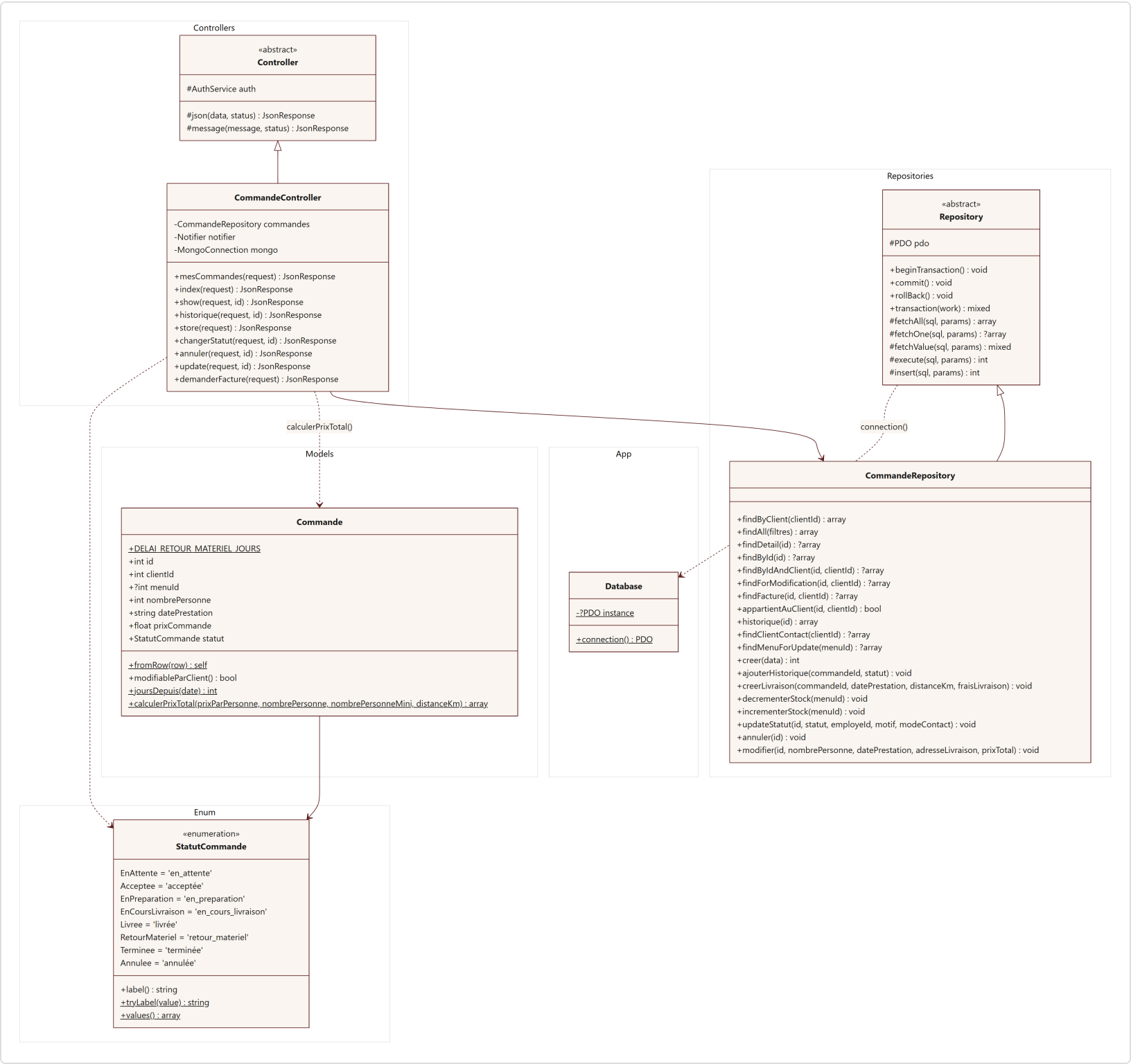


Diagramme de classes — domaine « commande » détaillé : MVC du contrôleur à la base (visibilités + public, # protégé, - privé ; souligné = statique).

Diagramme de séquence — Connexion — Parcours entre le navigateur, le `Kernel`, `AuthController`, `UtilisateurRepository`, le modèle `Utilisateur`, `PasswordPolicy` (`password_verify`) et `AuthService` (session), avec les trois issues possibles : identifiants invalides (401), compte désactivé (403), connexion valide.

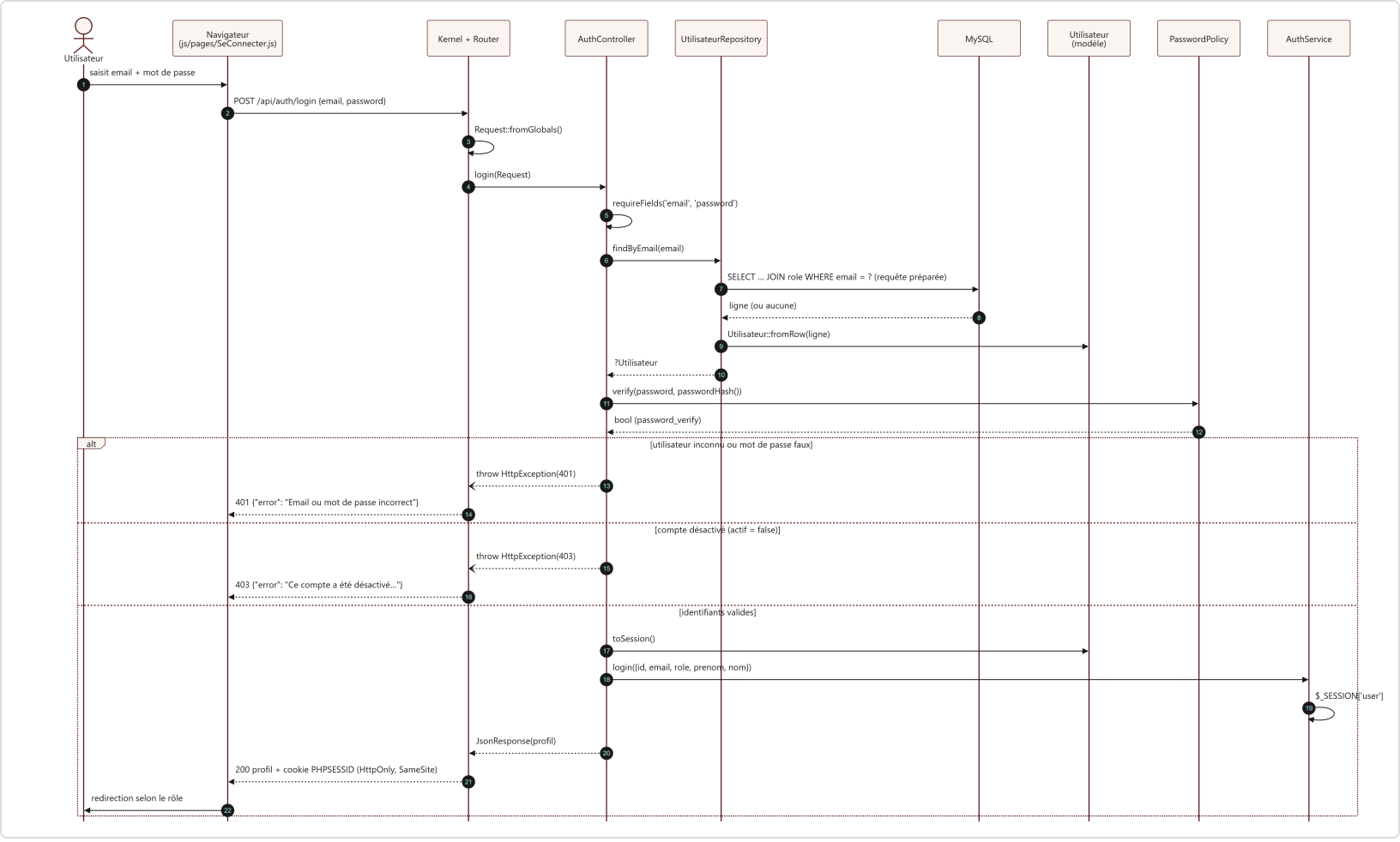


Diagramme de séquence — parcours de connexion sécurisé, au niveau des classes.

Diagramme de séquence — Commande — Parcours critique de la validation du panier à la création de la commande : transaction SQL portée par le repository, verrouillage et décrément du stock, calcul de la tarification par le modèle `Commande`, email de confirmation, puis écriture de la trace dans MongoDB pour les statistiques.

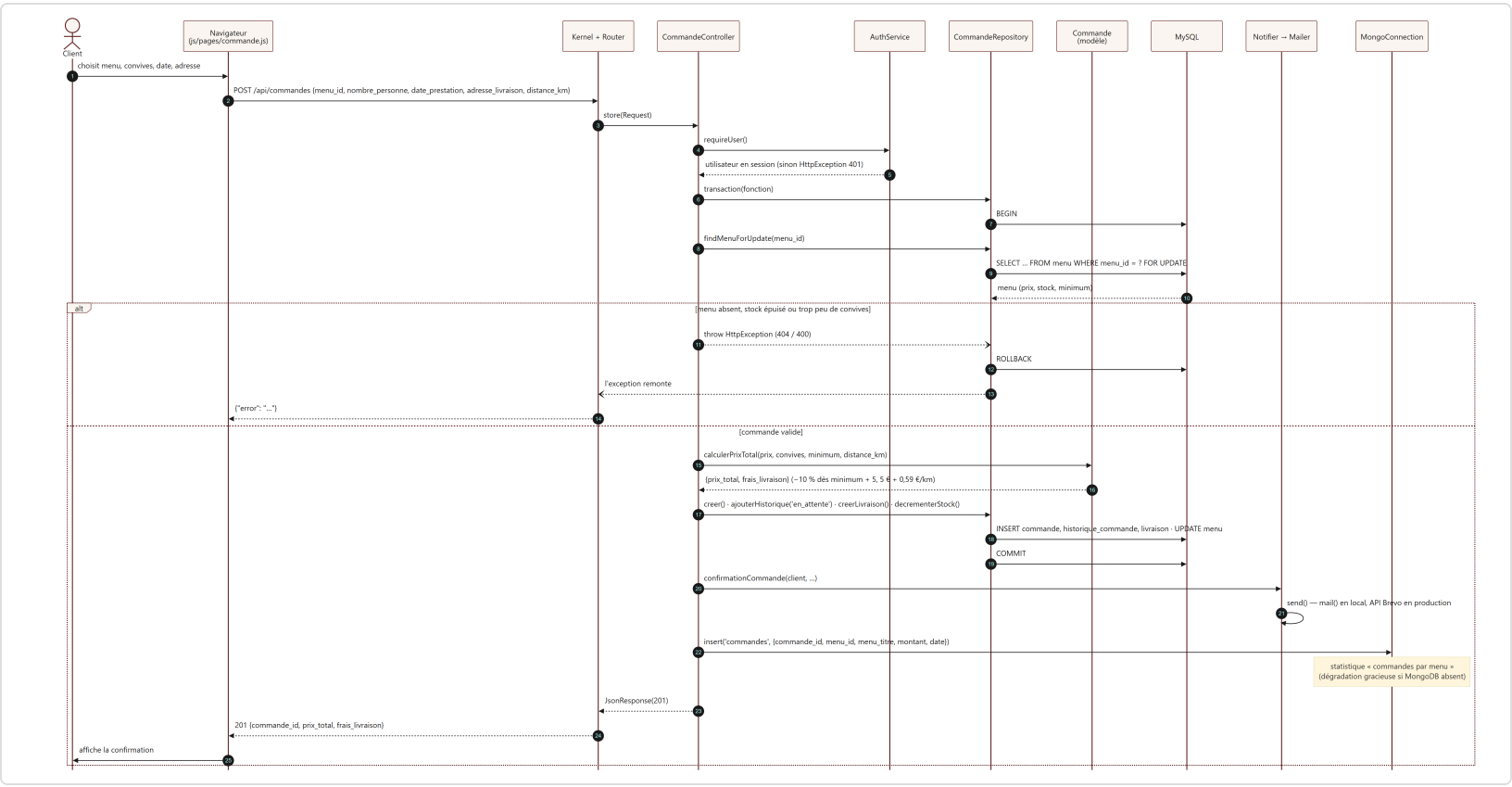


Diagramme de séquence — création d'une commande (transaction, règles métier, double persistance).

Partie 3 — Recherche & veille technologique (documentation anglophone)

3.1 Situation de travail ayant nécessité une recherche anglophone

Lors de l'implémentation du hachage des mots de passe avec `password_hash()` et l'algorithme `PASSWORD_BCRYPT`, il a fallu déterminer la valeur du **facteur de coût** (`cost`) : ce paramètre fixe la difficulté de calcul du hachage. Une valeur trop basse rend le hachage vulnérable à une attaque par force brute sur une base volée ; une valeur trop haute ralentit chaque connexion et charge inutilement le serveur. La documentation officielle de PHP, disponible uniquement en anglais dans sa version de référence, précise la démarche à suivre.

Source : *PHP Manual —* `password_hash` , php.net/manual/en/function.password-hash.php (section « Supported options for PASSWORD_BCRYPT » et « Notes »).

3.2 Extrait anglophone et traduction

Extrait original (anglais)

> *« `cost` (int) — which denotes the algorithmic cost that should be used. Examples of these values can be found on the `crypt()` page. If omitted, a default value of 12 will be used. This is a good baseline cost, but it should be adjusted depending on hardware used. »*
>
> *« Note: It is recommended to test this function on the machine used, adjusting the cost parameter(s) so that execution of the function takes less than 350 milliseconds for interactive logins. »*

Traduction en français

> « `cost` (entier) — désigne le coût algorithmique à utiliser. Des exemples de ces valeurs se trouvent sur la page `crypt()` . S'il est omis, une valeur par défaut de 12 est utilisée. C'est un bon coût de référence, mais il doit être ajusté selon le matériel utilisé.
>
> Note : il est recommandé de tester cette fonction sur la machine utilisée, en ajustant le paramètre de coût de sorte que son exécution prenne moins de 350 millisecondes pour les connexions interactives. »

3.3 Décision retenue

J'ai fixé explicitement le coût à 12 dans la constante `BCRYPT_COST` de la classe `PasswordPolicy` (`api/src/Security/PasswordPolicy.php`), utilisée par sa méthode `hash()` . Ce choix correspond au coût de référence recommandé par PHP ; sur l'environnement de test, un hachage prend environ 250 ms, bien en dessous du seuil de 350 ms préconisé pour ne pas dégrader l'expérience de connexion, tout en restant assez lent pour limiter fortement le débit d'un attaquant. La valeur est écrite en dur plutôt que laissée implicite, afin que le comportement reste stable même si la valeur par défaut de PHP évolue dans une version ultérieure.

Partie 4 — Conformité RGPD et accessibilité RGAA

4.1 Conformité RGPD et pseudonymisation

Bases légales des traitements — Consentement explicite à l'inscription (case à cocher non pré-cochée, tracée en base) ; exécution d'un contrat pour la prise de commande ; intérêt légitime pour les emails transactionnels. Aucun cookie tiers (analytics, publicité).

Droits implémentés — Accès (Art. 15) et rectification (Art. 16) via l'espace profil ; portabilité (Art. 20) par export JSON téléchargeable ; effacement (Art. 17).

Conflit droit à l'effacement (Art. 17) / obligation comptable (Code de commerce Art. L123-22) — Les commandes doivent être conservées 10 ans ; elles ne peuvent donc pas être supprimées. La solution retenue est la **pseudonymisation** : lors d'une demande de suppression, le compte est désactivé et les données nominatives (email, nom, prénom, téléphone, adresse, ville) sont remplacées par des valeurs anonymes. L'historique d'achat subsiste à des fins comptables sans permettre d'identifier le client. Les comptes employés et administrateurs sont exclus de cette route.

4.2 Accessibilité (RGAA / WCAG 2.1 niveau AA)

- **Navigation au clavier** — Prise en charge complète de `Tab`, `Entrée` et `Échap` sur l'ensemble des composants interactifs (modales, menu mobile, formulaires).
- **Sémantique ARIA** — Le bouton hamburger porte un `aria-label` explicite et un `aria-expanded` mis à jour dynamiquement en JavaScript pour annoncer l'état ouvert / fermé aux lecteurs d'écran.
- **Structure HTML sémantique** — Chaque page utilise un unique `<header>` , `<main>` et `<footer>` ; les formulaires associent chaque `<label for>` à l' `id` de son champ ; les images décoratives portent un `alt=""` , les images informatives une description.

- **Contrastes** — Le texte principal (`#2C1810`) sur le fond crème (`#F7F1E8`) atteint un ratio de **15,02:1**, très au-delà du minimum de 4,5:1 exigé par le niveau AA et du seuil de 7:1 du niveau AAA. Le bordeaux (`#5C1A1A`) sur crème atteint 11,56:1, également conforme AAA.

Dossier réalisé dans le cadre du TP Développeur Web et Web Mobile — Studi, session 2026.